

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – Searching



Nayla Isma Uzlifatul Afroh

109082500158

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pada modul ini, kita mempelajari pencarian data yang lebih spesifik dalam array dan struct. Berbeda dengan pencarian nilai ekstrim yang selalu ditemukan, pencarian biasa kadang data yang dicari tidak ada. Oleh karena itu, kita perlu memahami algoritma yang efisien agar program bisa langsung memberi tahu apakah data ada atau tidak, dan di posisi mana data tersebut berada.

Salah satu algoritma yang diperkenalkan adalah Sequential Search. Pada metode ini, program memeriksa setiap elemen dari awal sampai akhir secara berurutan. Proses pencarian akan berhenti ketika data ditemukan atau seluruh array telah dicek. Sequential search mudah dipahami dan diterapkan, terutama untuk array yang ukurannya kecil. Namun, jika jumlah data banyak, metode ini bisa memakan waktu karena harus mengecek satu per satu semua elemen.

Selain itu, ada juga Binary Search. Algoritma ini lebih cepat dibanding sequential search, tetapi syaratnya data harus terurut ascending. Binary search bekerja dengan membagi array menjadi dua bagian pada tiap langkah, kemudian menentukan apakah data yang dicari ada di bagian kiri atau kanan. Hal ini membuat jumlah perbandingan lebih sedikit dan pencarian lebih efisien. Binary search tidak bisa digunakan jika data terurut descending, kecuali algoritma disesuaikan.

Modul ini juga menjelaskan pencarian pada array bertipe struct, misalnya array yang menyimpan data mahasiswa dengan atribut nama, NIM, dan IPK. Konsepnya mirip pencarian di array biasa, tapi kita harus memperhatikan field yang digunakan sebagai kunci. Untuk binary search, array harus diurutkan berdasarkan field kunci, misal NIM mahasiswa. Jika field kunci berbeda atau array belum terurut, kita tetap bisa menggunakan sequential search. Dengan memahami ketiga konsep inisequential search, binary search, dan pencarian di array struct kita bisa memilih algoritma yang tepat sesuai jenis data dan ukuran array. Modul ini membantu kita membangun program yang lebih efisien, terstruktur, dan siap diterapkan untuk kasus nyata seperti mencari mahasiswa, calon ketua RT, atau data lain yang disimpan dalam array maupun struct

B. Guided

1. Guided 1

Source code:

```
guided1.go U X
C: > Users > Asus > OneDrive - Telkom University > Documents > ALPRO 2 > Repository Laprak (GitHub) > 109082500158_Na

1 package main
2
3 import "fmt"
4
5 type arrData [5]string //tipe data alias
6
7 func seqSearch(arr arrData, binatangCari string) int {
8     var found int = -1
9     for i := 0; i < len(arr); i++ {
10         if arr[i] == binatangCari {
11             found = i
12             break
13         }
14     }
15     return found
16 }
17
18 func main() {
19     var arrBinatang arrData //array tipe string
20
21     for i := 0; i < len(arrBinatang); i++ {
22         fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)
23         fmt.Scan(&arrBinatang[i])
24     }
25     fmt.Println()
26
27     var binatangCari string
28     fmt.Print("Masukkan nama binatang yang mau dicari : ")
29     fmt.Scan(&binatangCari)
30
31     var idxCari int
32     idxCari = seqSearch(arrBinatang, binatangCari)
33
34     if idxCari > -1 {
35         fmt.Printf("Data %s ditemukan pada indeks ke-%d!", binatangCari, idxCari)
36     } else if idxCari == -1 {
37         fmt.Printf("Data %s tidak ditemukan!", binatangCari)
38     }
39
40 }
```

Output :

```
PS C:\Users\Asus> go run "c:\Users\Asus\OneDrive - Telkom University\Documents\ALPRO 2\Repository Laprak (GitHub)\109082500158_Nayla-Isma-Uzlifatul-Afroh\week 11-modul 12\guided\guided1.go"
Masukkan data binatang indeks ke-0 : kucing
Masukkan data binatang indeks ke-1 : anjing
Masukkan data binatang indeks ke-2 : ayam
Masukkan data binatang indeks ke-3 : kambing
Masukkan data binatang indeks ke-4 : sapi

Masukkan nama binatang yang mau dicari : kambing
Data kambing ditemukan pada indeks ke-3!
PS C:\Users\Asus>
```

Penjelasan :

Di program ini, kita pakai array buat nyimpen nama-nama binatang. Array-nya ukurannya tetap, jadi cuma bisa 5 elemen aja. Fungsi seqSearch itu buat nyari binatang yang kita input, mulai dari indeks pertama sampe terakhir. Kalau ketemu, dia bakal ngasih tau indeksnya, kalau nggak ada, keluarin -1. Intinya, ini nunjukin gimana pencarian linear di array kerja, gampang dimengerti buat belajar dasar array.

2. guided 2

Source code:

```
> Users > Asus > OneDrive - Telkom University > Documents > ALPRO 2 > Repository Laprak (GitHub)
1  package main
2
3  import "fmt"
4
5  // fungsi binary search
6  func binarySearch(data []int, x int) int {
7
8      // batas kiri array
9      kiri := 0
10
11      // batas kanan array
12      kanan := len(data) - 1
13
14      // perulangan selama kiri <= kanan
15      for kiri <= kanan {
16
17          // mencari index tengah
18          tengah := (kiri + kanan) / 2
19
20          // jika data ditemukan
21          if data[tengah] == x {
22              return tengah
23          }
24          // jika x lebih besar, cari ke kanan
25          } else if data[tengah] < x {
26              kiri = tengah + 1
27          }
28          // jika x lebih kecil, cari ke kiri
29          } else {
30              kanan = tengah - 1
31          }
32      }
33
34      // jika data tidak ditemukan
35      return -1
36  }
```

```
37
38 func main() {
39
40     var n int
41     var cari int
42
43     // input jumlah data
44     fmt.Print("Jumlah data: ")
45     fmt.Scan(&n)
46
47     // membuat array sesuai jumlah data
48     data := make([]int, n)
49
50     // input isi array
51     fmt.Println("Masukkan data terurut:")
52     for i := 0; i < n; i++ {
53         fmt.Scan(&data[i])
54     }
55
56     // input angka yang dicari
57     fmt.Print("Angka yang dicari: ")
58     fmt.Scan(&cari)
59
60     // memanggil fungsi binary search
61     hasil := binarySearch(data, cari)
62
63     // output hasil pencarian
64     if hasil != -1 {
65         fmt.Println("Data ditemukan di index:", hasil)
66     } else {
67         fmt.Println("Data tidak ditemukan")
68     }
69 }
```

Output :

```
PS C:\Users\Asus> go run "c:\Users\Asus\OneDrive - Telkom University\Documents\1-Afroh\week 11-modul 12\guided\guided2.go"
Jumlah data: 5
Masukkan data terurut:
10
20
30
40
50
Angka yang dicari: 30
Data ditemukan di index: 2
PS C:\Users\Asus> █
```

Penjelasan:

Di sini kita pakai **slice**, jadi datanya bisa lebih fleksibel. Tapi, syaratnya data harus terurut dulu. Fungsi `binarySearch` bikin proses nyari angka lebih cepat dibanding `linear search`, karena tiap langkah dia ngecek tengah array, terus tentuin maju ke kiri atau kanan. Kalau ketemu, muncul indeksnya, kalau nggak ketemu, muncul pesan "Data tidak ditemukan". Konsep ini nunjukin efisiensi pencarian biner dibanding linear, cocok buat array besar.

3. guided 3

```
guided1.go 2, U    guided2.go 1, U    guided3.go 1, U X
C:\Users\Asus\OneDrive - Telkom University\Documents\ALPRO 2\Repository Laprak (GitHub)\109082500158_Nayla-Isma-Uz
1  package main
2
3  import "fmt"
4
5  type mahasiswa struct {
6      nama string
7      nim  int
8      IPK  float64
9  }
10
11 type arrMahasiswa [5]mahasiswa
12
13 func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
14     var found int = -1
15     var med int
16     var kr int = 0
17     var kn int = len(arrData) - 1
18     for kr <= kn && found == -1 {
19         med = (kr + kn) / 2
20         if nimDicari < arrData[med].nim {
21             kn = med - 1
22         } else if nimDicari > arrData[med].nim {
23             kr = med + 1
24         } else {
25             found = med
26         }
27     }
28     return found
29 }
30
31 func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
32     var found int = -1
33     for i := 0; i < len(arrData); i++ {
34         if arrData[i].nim == nimDicari {
35             found = i
36             break
37         }
38     }
39     return found
40 }
41
42 func main() {
43     var mahasiswa06 arrMahasiswa
44     var nimDicari int
45
46     fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---")
47     for i := 0; i < len(mahasiswa06); i++ {
48         fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks ke-%d ===\n", i)
```



```

48     fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks ke-%d ===\n", i)
49     fmt.Print("Masukkan nama : ")
50     fmt.Scan(&mahasiswa06[i].nama)
51     fmt.Print("Masukkan NIM : ")
52     fmt.Scan(&mahasiswa06[i].nim)
53     fmt.Print("Masukkan IPK : ")
54     fmt.Scan(&mahasiswa06[i].IPK)
55     fmt.Println("-----")
56 }
57 fmt.Println()
58
59 fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
60 fmt.Scan(&nimDicari)
61 fmt.Println()
62
63 var idxHasilCariSeqSearch int
64 idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06, nimDicari)
65
66 fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
67 if idxHasilCariSeqSearch > -1 {
68     fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
69     fmt.Println("Berikut Detail Datanya : ")
70     fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariSeqSearch].nama)
71     fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCariSeqSearch].nim)
72     fmt.Printf("IPK : %.2f\n", mahasiswa06[idxHasilCariSeqSearch].IPK)
73 } else if idxHasilCariSeqSearch == -1 {
74     fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!", nimDicari)
75 }
76 fmt.Println()
77
78 var idxHasilCariBinSearch int
79 idxHasilCariBinSearch = binSearchStruct(mahasiswa06, nimDicari)
80
81 fmt.Println("== HASIL BINARY SEARCH ==")
82 if idxHasilCariBinSearch > -1 {
83     fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
84     fmt.Println("Berikut Detail Datanya : ")
85     fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariBinSearch].nama)
86     fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCariBinSearch].nim)
87     fmt.Printf("IPK : %.2f\n", mahasiswa06[idxHasilCariBinSearch].IPK)
88 } else if idxHasilCariBinSearch == -1 {
89     fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!", nimDicari)
90 }
91 }

```

Output :

```

PS C:\Users\Asus> go run "c:\Users\Asus\OneDrive - Telkom University\Documents\ALP
--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : nayla
Masukkan NIM : 109082500158
Masukkan IPK : 4.00
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : isma
Masukkan NIM : 109082500159
Masukkan IPK : 4.00
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : uzlifatul
Masukkan NIM : 109082500160
Masukkan IPK : 3.90

```

```

-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : uzlifatul
Masukkan NIM : 109082500160
Masukkan IPK : 3.90
-----

=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : afroh
Masukkan NIM : 109082500161
Masukkan IPK : 4.00
-----

=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : lala
Masukkan NIM : 109082500162
Masukkan IPK : 4.00
-----

Masukkan NIM mahasiswa yang ingin dicari : 109082500158

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 109082500158 ditemukan pada array di indeks ke-0!
Berikut Detail Datanya :
Nama : nayla
NIM : 109082500158
IPK : 4.00

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 109082500158 ditemukan pada array di indeks ke-0!
Berikut Detail Datanya :
Nama : nayla
NIM : 109082500158
IPK : 4.00

```

Penjelasan:

Di praktikum ini, kita belajar nyari data mahasiswa yang disimpan dalam **struct**. Setiap mahasiswa punya nama, NIM, dan IPK. Program pakai dua metode pencarian: sequential search dan binary search.

Sequential search ngecek satu per satu mahasiswa sampai ketemu NIM yang dicari. Binary search lebih cepat karena data NIM harus **urut**, lalu dicari dengan membagi array di tengah tiap langkah. Hasil kedua metode ini sama-sama ngasih tahu indeks mahasiswa plus detail datanya.

Praktikum ini bikin kita paham gimana algoritma pencarian bisa diterapin ke data yang lebih kompleks dibanding array biasa, sekaligus ngerasain perbedaan efisiensi antara sequential dan binary search.

C. Unguided

1. Soal Unguided 1

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda adalah membuat program untuk mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program **pilkart** yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 sampai 20. Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris mencetak data para calon yang mendapatkan suara.

```

1  package main
2
3  import "fmt"
4
5  func main() {
6      var suara [21]int
7      var x int
8      masuk := 0
9      sah := 0
10
11     fmt.Scan(&x)
12     for x != 0 {
13         masuk++
14
15         if x >= 1 && x <= 20 {
16             suara[x]++
17             sah++
18         }
19
20         fmt.Scan(&x)
21     }
22
23     fmt.Println("Suara masuk:", masuk)
24     fmt.Println("Suara sah:", sah)
25
26     for i := 1; i <= 20; i++ {
27         if suara[i] > 0 {
28             fmt.Printf("%d: %d\n", i, suara[i])
29         }
30     }
31 }

```

Output:

```

PS C:\Users\Asus> go run "c:\Users\Asus\OneDrive - Telkom University\Docum
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS C:\Users\Asus>

```

Penjelasan :

Program ini digunakan untuk membaca data suara dalam pemilihan ketua RT. Setiap data yang masuk dihitung sebagai suara masuk selama belum ditemukan angka 0. Data suara yang sah hanya bernilai 1 sampai 20, sedangkan angka di luar rentang tersebut dianggap tidak valid. Setiap suara sah akan disimpan ke dalam array berdasarkan nomor calon. Setelah seluruh input selesai dibaca, program menampilkan jumlah suara masuk, jumlah suara sah, serta daftar calon yang memperoleh suara.

2. Soal Unguided 2

Berdasarkan program sebelumnya, buatlah program **pilkart** yang mencari siapa pemenang pemilihan ketua RT. Selain itu, tentukan juga wakil ketua RT, yaitu calon yang mendapatkan suara terbanyak kedua.

Jika beberapa calon mendapatkan suara terbanyak yang sama, maka ketua terpilih adalah calon dengan nomor peserta yang paling kecil dan wakilnya adalah calon dengan nomor peserta terkecil berikutnya.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 sampai 20. Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

Jawaban :

```
1  package main
2
3  import "fmt"
4
5  func main() {
6      var suara [21]int
7      var x int
8      masuk := 0
9      sah := 0
10
11     fmt.Scan(&x)
12     for x != 0 {
13         masuk++
14
15         if x >= 1 && x <= 20 {
16             suara[x]++
17             sah++
18         }
19
20         fmt.Scan(&x)
21     }
22
23     ketua := -1
24     wakil := -1
25
26     for i := 1; i <= 20; i++ {
27         if suara[i] > 0 {
28             if ketua == -1 || suara[i] > suara[ketua] || (suara[i] == suara[ketua] && i < ketua) {
29                 wakil = ketua
30                 ketua = i
31             } else if wakil == -1 || suara[i] > suara[wakil] || (suara[i] == suara[wakil] && i < wakil) {
32                 wakil = i
33             }
34         }
35     }
36
37     fmt.Println("Suara masuk:", masuk)
38     fmt.Println("Suara sah:", sah)
39     fmt.Println("Ketua RT:", ketua)
40     fmt.Println("Wakil ketua:", wakil)
41 }
```

Output :

```
PS C:\Users\Asus> go run "c:\Users\Asus\OneDrive - Telkom University\Doc
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS C:\Users\Asus> go run "c:\Users\Asus\OneDrive - Telkom University\Doc
█
```

Penjelasan:

Program ini merupakan pengembangan dari program sebelumnya. Selain membaca, memvalidasi, dan menghitung suara, program juga menentukan calon yang menjadi ketua RT dan wakil ketua RT. Calon dengan jumlah suara terbanyak akan menjadi ketua RT, sedangkan calon dengan suara terbanyak kedua menjadi wakil ketua. Jika terdapat jumlah suara yang sama, maka calon dengan nomor peserta lebih kecil akan diprioritaskan. Pada contoh, calon nomor 3 dan 19 sama-sama memperoleh 2 suara, sehingga calon nomor 3 menjadi ketua dan calon nomor 19 menjadi wakil ketua.

3. Soal Unguided 3

Diberikan **n** data integer positif dalam keadaan terurut membesar dan sebuah integer lain **k**. Tentukan apakah bilangan **k** tersebut ada dalam daftar bilangan yang diberikan. Jika ada, tampilkan indeksnya. Jika tidak ada, tampilkan tulisan **"TIDAK ADA"**.

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu **n** dan **k**. Nilai **n** menyatakan banyaknya data, dengan ketentuan $1 < n \leq 1000000$, sedangkan **k** adalah bilangan yang ingin dicari. Baris kedua berisi **n** buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari dalam kumpulan data yang diberikan. Posisi data dihitung mulai dari indeks 0. Jika data **k** tidak ditemukan, maka keluaran yang ditampilkan adalah **"TIDAK ADA"**.

Program yang dibuat harus menggunakan subprogram sesuai kerangka yang telah diberikan pada modul.

Jawaban :

```
C: > Users > Asus > OneDrive - Telkom University > Documents > ALPRO 2 > Repository Laprak (GitHub) > 109082500158_Nayla-
1  package main
2
3  import "fmt"
4
5  const NMAX = 1000000
6
7  var data [NMAX]int
8
9  func main() {
10     var n, k int
11
12     fmt.Scan(&n, &k)
13     isiArray(n)
14
15     hasil := posisi(n, k)
16
17     if hasil == -1 {
18         fmt.Println("TIDAK ADA")
19     } else {
20         fmt.Println(hasil)
21     }
22 }
23
24 func isiArray(n int) {
25     for i := 0; i < n; i++ {
26         fmt.Scan(&data[i])
27     }
28 }
29
30 func posisi(n, k int) int {
31     kiri := 0
32     kanan := n - 1
33
34     for kiri <= kanan {
35         tengah := (kiri + kanan) / 2
36
37         if data[tengah] == k {
38             return tengah
39         } else if data[tengah] < k {
40             kiri = tengah + 1
41         } else {
42             kanan = tengah - 1
43         }
44     }
45
46     return -1
47 }
```


Output :

```
PS C:\Users\Asus> go run "c:\Users\Asus\OneDrive - Telkom Universi
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\Users\Asus> go run "c:\Users\Asus\OneDrive - Telkom Universi
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
PS C:\Users\Asus> go run "c:\Users\Asus\OneDrive - Telkom Universi
[]
```

Penjelasan :

Program ini digunakan untuk mencari posisi suatu bilangan dalam array yang sudah terurut membesar. Data dimasukkan melalui prosedur isiArray, sedangkan proses pencarian dilakukan oleh fungsi posisi. Karena data sudah dalam keadaan terurut, pencarian dilakukan menggunakan binary search. Algoritma ini membandingkan nilai yang dicari dengan elemen tengah array. Jika nilai tengah lebih kecil dari nilai yang dicari, pencarian dilanjutkan ke bagian kanan. Jika nilai tengah lebih besar, pencarian dilanjutkan ke bagian kiri. Jika data ditemukan, fungsi mengembalikan indeksnya. Jika tidak ditemukan, fungsi mengembalikan nilai -1 dan program menampilkan "TIDAK ADA".

D.Kesimpulan

Dari praktikum Modul 12 ini dapat disimpulkan bahwa proses pencarian data dapat dilakukan dengan beberapa metode, yaitu **sequential search** dan **binary search**. Sequential search melakukan pencarian dengan mengecek data satu per satu dari awal sampai data ditemukan atau seluruh data selesai diperiksa. Metode ini dapat digunakan pada data yang belum terurut, tetapi kurang efisien jika jumlah data sangat banyak.

Binary search digunakan untuk mencari data pada array yang sudah terurut. Metode ini lebih efisien karena pencarian dilakukan dengan membagi rentang data menjadi dua bagian, sehingga jumlah pengecekan menjadi lebih sedikit. Namun, binary search hanya dapat berjalan dengan benar jika data sudah dalam keadaan terurut sesuai kebutuhan pencarian.

Melalui soal unguided, dapat dipahami bahwa konsep searching tidak hanya digunakan untuk mencari posisi suatu data, tetapi juga dapat diterapkan dalam pengolahan data lain, seperti validasi suara, menghitung frekuensi kemunculan data, serta menentukan data dengan nilai terbesar dan terbesar kedua. Dengan memahami algoritma searching, proses

pencarian dan pengolahan data dalam program dapat dilakukan secara lebih terstruktur, cepat, dan tepat.